Creating Union Types

I want to start with a concept we've already seen, but which I want to talk about in more depth because it winds up being important for understanding other TypeScript techniques. The feature is called a *union type*, and we used it to create the type checking for our action types in our various reducers in Chapter 11, *Managing State in React* and in Chapter 12, *Using Redux to Manage State*.

You create a union type by combining existing types with the pipe operator (|). The existing types can be built-in or specific to your code. One of our union types, for example, looks like this:

chapter_13/01/app/javascript/contexts/venue_types.ts

```
export type VenueAction =
  | ClearHolds
  | HoldTicket
  | InitFromProps
  | SetTicketToBuy
  | SetTickets
  | UnholdTicket
```

We're using the `type` keyword here to assign a type to a new name, which we then use in our method signature:

chapter_13/01/app/javascript/contexts/venue_context.ts

```
export const venueReducer = (
  state: VenueState = initialState,
  action: VenueAction
): VenueState => {
```

We don't have to use the `type` keyword, though. We could use the union type inline:

```
export const venueReducer = (
  state: VenueState = initialState,
  action: ClearHolds
  | HoldTicket
```